Stability and Structural Validity of Learned Node Importance in Citation Networks

Niran Czuckermann · **Shahar Bar** Advisor: Dr. Dvora Toledano-Kitai

Software Engineering Department, Braude College of Engineering · Final Project 26-2-R-2

Code, data & live demo

KEY FINDING

Combining centralities with a Random Forest makes the ranking more robust than PageRank — the measure it learns — against **added** citations, but not against **missing** ones.

61% vs 42%
of the top-10 % papers stay on top: forest vs PageRank, after adding 20 % citations

0.83 vs 0.86
Spearman with the original ranking after removing 20 % of citations: forest vs PageRank

270
damaged networks: 3 kinds of damage × 3 levels × 30 runs

1 The question

Centrality measures rank papers by importance, but citation data is incomplete and noisy. A small change in the network can reorder the ranking.

Is a learned combination of measures more stable than any single one?

- Are the most important papers also the most stable?
- Which papers look important but are fragile?

2 The data

19,717 papers on diabetes

44,324 citations

PubMed-Diabetes citation network. Cleaning removed 3 self-citations and 11 mutual citations: the result has no citation loops and is one connected network.

3 Damaging the network

− **Remove**
5, 10 or 20 % of citations — missing data

\+ **Add, newer → older**
fake citations that are still possible in time

± **Add at random**
the Phase A design; creates impossible citation loops

4 How we measure stability

1 **Centralities** — degree, betweenness, closeness, PageRank, k-core

2 **Frozen scaler** — Min-Max, fitted once on the original network

3 **Random Forest** — learns PageRank from the other four measures

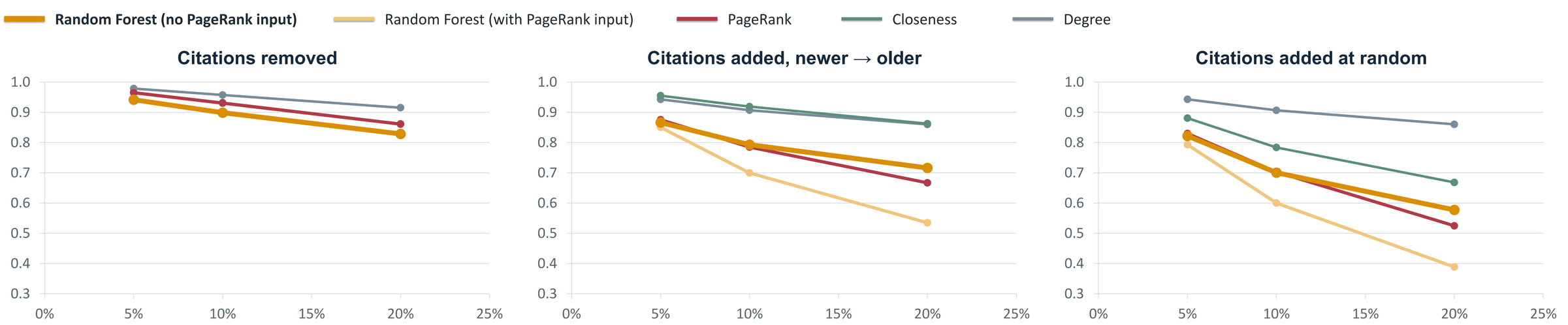
4 **Damage** — remove or add citations; 30 runs per setting

5 **Re-score** — same scaler and same forest — no retraining

6 **Compare** — Spearman, Kendall, top-10 % kept, per-paper CV

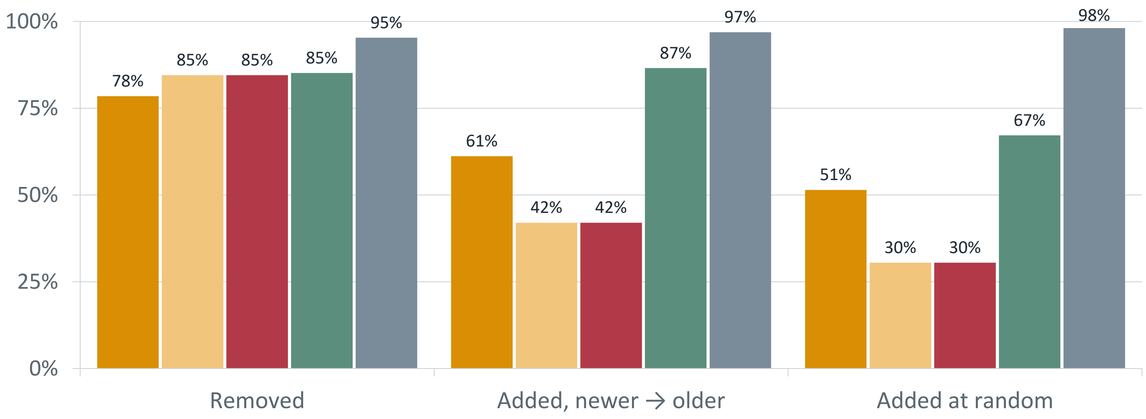
Two forests are compared with the five single measures. The forest that also gets PageRank as an input (the original design) just copies it — 99.8 % of its feature importance — so the forest without it is the main model.

5 How much does the ranking change?



Mean Spearman correlation between the original ranking and the damaged one (30 runs per point; the runs agree closely). The forest beats PageRank when citations are added, not when they are removed. Left panel: PageRank, closeness and the forest with PageRank input lie on top of each other. k-core (not shown) behaves like degree.

6 Do the top papers stay on top?



Share of runs in which a top-10 % paper is still in the top 10 %, at 20 % damage. Under added noise most of PageRank's top papers are fragile; the forest keeps more of them.

7 The hypotheses

Hypothesis	Verdict	What we found
H1 Learned rankings are more stable than single measures	Partly	Better than PageRank under added noise; worse under removal; never better than degree or k-core
H2 The forest absorbs a shock to one feature	Not supported	The forest with PageRank input relies on it and collapses with it — less stable than PageRank itself
H3 Key papers keep a high rank	Supported	Core papers stay in the top 10 % in 65–94 % of runs (adjusted core rule)

8 What the data taught us

- **Ties look like stability.** Betweenness is 0 for 91 % of papers, so over all papers it looks the most stable — yet its top 10 % is the least stable. Compare at the top.
- **Trees cannot extrapolate.** After damage, up to 60 % of the inputs leave the training range and get one value. Part of the forest's stability is blindness.
- **One citation, 1,545 places.** Paper 15004560 fell from rank 67 to 1,612 after gaining one reference: its betweenness jumped from 0.

Conclusion

A learned combination of centralities is more robust than PageRank against noisy citations, especially at the top of the ranking. The gain does not extend to missing citations, and simple local measures stay the most stable.

Limits: one dataset, one target (PageRank), one model type; trees cannot extrapolate.

Data: PubMed-Diabetes (Namata et al., 2012; LINQS). Tools: Python, NetworkX, igraph, scikit-learn. Code, notebooks and interactive demo: github.com/ShaharBar99/Final-Project